快速排序与划分暴力解模板

408 | 数据结构 · 双指针一趟划分、2016 真题全排暴力解 vs 快速选择最优解

一、快排核心零件：一趟双指针划分 Partition（手写笔记第 1 页）

快速排序的划分本质是双指针左右夹逼填坑，三条关键记忆点：

1. 双指针：low 和 high 两个指针向中间靠拢；
2. 从右往左初始：枢轴取自 a[low]，此时左侧形成空位，必须先从右边 high-- 寻找小于枢轴的元素；
3. 每次发生互换填坑：右边找到小数填入左侧空位，左边找到大数填入右侧空位，相遇处填回枢轴。

```
/* 通用母板：一趟双指针划分（左右交替填坑），时间 O(n)，空间 O(1) */
int Partition(int a[], int low, int high) {
    int pivot = a[low];                 /* 1. 枢轴先取出暂存，low 处为空位 */
    while (low < high) {
        while (low < high && a[high] >= pivot) high--;  /* 2. 从右往左找小数 */
        a[low] = a[high];                               /* 小数填入左侧空位 */
        while (low < high && a[low] <= pivot) low++;    /* 3. 从左往右找大数 */
        a[high] = a[low];                               /* 大数填入右侧空位 */
    }
    a[low] = pivot;                     /* 4. 相遇处填回枢轴 */
    return low;                         /* 返回枢轴最终下标 */
}

/* 完整快速排序递归模板（全排暴力解基础母板） */
void QuickSort(int a[], int low, int high) {
    if (low < high) {
        int pivotpos = Partition(a, low, high);
        QuickSort(a, low, pivotpos - 1);        /* 递归左半段 */
        QuickSort(a, pivotpos + 1, high);       /* 递归右半段 */
    }
}
```

二、2016 统考真题：暴力解法 vs 最优解法

题意要求：将含 n 个正整数的数组划分为 A_1 和 A_2（个数为 n_1, n_2，和为 S_1, S_2），使 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。编程输出 $|S_1 - S_2|$。

- 数学本质：个数尽量均分（$n_1 = \lfloor n/2 \rfloor$），且小的全部放在 A_1，大的全部放在 A_2。

1. 考场通杀暴力解（全排后切半，稳拿 **11~12** 分）

- 核心思想：既然要把最小的 $n/2$ 个数分给 A_1、其余分给 A_2，最直接无脑的办法就是直接调用排序算法将全数组升序排好！排完后直接将前 $\lfloor n/2 \rfloor$ 个数划入 A_1，后半部分划入 A_2。
- 优点：思维成本为 0，绝对不会写错单侧边界！三段式答题写满，评卷老师直接给 11~12 分（仅扣少量时间复杂度分）。

```
/* 2016 真题全排暴力解：全排序后直接对半切，时间 O(n log n)，空间 O(log n) */
int SplitArray_Brute(int a[], int n) {
    QuickSort(a, 0, n - 1);             /* 第一步：直接对整个数组做完整快排 */
    int k = n / 2;
    int s1 = 0, s2 = 0, i;
    for (i = 0; i < k; i++) s1 += a[i]; /* 前 k 个小数累加进 A1 */
    for (i = k; i < n; i++) s2 += a[i]; /* 后面大数累加进 A2 */
    return s2 - s1;                     /* 返回差值 */
}
```

2. 考场进阶最优解（快速选择 **QuickSelect**，冲刺 **15** 分满分）

- 优化原理：无需将所有数完全排好序，只需要找到第 $k = \lfloor n/2 \rfloor$ 小的元素即可。利用 Partition 划分，每次只向包含目标分界点的那一侧继续推进，另一侧整块丢弃！
- 时间由 $O(n \log n)$ 降至 $O(n)$，空间由 $O(\log n)$ 优化为 $O(1)$。

```
/* 2016 真题最优解：快速选择单侧迭代推进，平均时间 O(n)，空间 O(1) */
int SplitArray_Optimal(int a[], int n) {
    int low = 0, high = n - 1, k = n / 2, p;
    while (low < high) {
        p = Partition(a, low, high);
        if (p == k - 1) break;          /* 枢轴正好就位于分界点，提前收工 */
```

```
        else if (p < k - 1) low = p + 1;/* 目标在右半侧，只在右侧推进 */
        else high = p - 1;              /* 目标在左半侧，只在左侧推进 */
    }
    int s1 = 0, s2 = 0, i;
    for (i = 0; i < k; i++) s1 += a[i];
    for (i = k; i < n; i++) s2 += a[i];
    return s2 - s1;
}
```

三、2016 真题暴力解答题标准示范（三段式拿 11~12 分）

第一问：算法设计思想

本算法采用先排序后切分的暴力求解思想。由题设条件，要使 $|n_1-n_2|$ 最小且 $|S_1-S_2|$ 最大，须将数组中较小的 $k=\lfloor n/2\rfloor$ 个元素分入子集 A_1，其余较大的元素分入子集 A_2。

算法首先调用快速排序函数对数组 A 进行升序排列；排序完成后，将前 k 个元素累加求和得 S_1，后 $n-k$ 个元素累加求和得 S_2；最后计算并返回 S_2-S_1。

第二问：核心代码

直接书写 `Partition` 函数、`QuickSort` 函数及 `SplitArray_Brute` 函数（见上文）。

第三问：时空复杂度

快速排序的平均时间复杂度为 $O(n\log n)$，统计两部分和各遍历一次为 $O(n)$，总时间复杂度为 $O(n\log n)$；递归工作栈平均深度为 $O(\log n)$，故空间复杂度为 $O(\log n)$。

四、快排与划分避坑红线

- 方向写反：枢轴取 `a[low]`，首步必须是 `high--`；写反成 `low++` 会在首轮直接覆盖掉 `a[high]` 的原值。
- 内层循环遗漏 `low < high`：若序列本身已经升序或降序，`high--` 会直接冲穿到负数下标导致段错误崩溃。
- 暴力解心态调整：大题 15 分，能写出最优解固然拿 15 分，但若考场紧张想不清快速选择边界，果断写全排暴力解，保底拿 11~12 分，省下时间做后面的计组/操作系统大题！